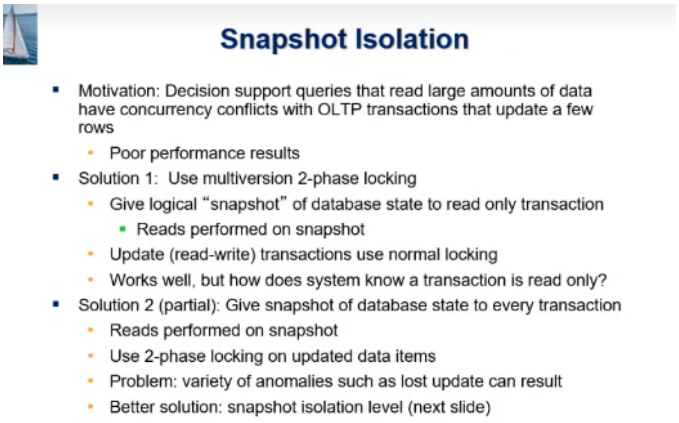
Weak Levels of consistency

For the lost update

- how to avoid deadlock → use wait mode (X lock from the beginning)
  - or we can use some management that group same data item together.
- If we use timestamp protocol → A will get roll back
  - what if R is not single row, multiple row → use optimistic approach call **snapshot isolation.**



  - snapshot isolation allows us to read the same data, optimistic approach assume that this 2 not gonna buy the same seat
    - worst case → buy the same seat → the one who commit first will get a seat
      - another roll back, get rejected, book again
- Task is done by the system.
- Some good DBMS has snapshot isolation
- snapshot isolation is a variation of read committed
  - you may notice that snapshot isolation is one of read committed

- **Read committed** — only committed records can be read.
  - Successive reads of record may return different (but committed) values.

✅ Weak Levels of Consistency

Weak Levels of Consistency

- **Degree-two consistency**: differs from two-phase locking in that S-locks may be released at any time, and locks may be acquired at any time
  - X-locks must be held till end of transaction
  - Serializability is not guaranteed, programmer must ensure that no erroneous database state will occur]
- **Cursor stability**:
  - For reads, each tuple is locked, read, and lock is immediately released
  - X-locks are held till end of transaction
  - Special case of degree-two consistency

Weak Levels of Consistency in SQL

- SQL allows non-serializable executions
  - **Serializable**: is the default
  - **Repeatable read**: allows only committed records to be read, and repeating a read should return the same value (so read locks should be retained)
    - However, the phantom phenomenon need not be prevented
      - T1 may see some records inserted by T2, but may not see others inserted by T2
  - **Read committed**: same as degree two consistency, but most systems implement it as cursor-stability
  - **Read uncommitted**: allows even uncommitted data to be read
- In most database systems, read committed is the default consistency level
  - Can be changed as database configuration parameter, or per transaction
    - **set isolation level serializable**

- sometimes, we need weaker level (snapshot is one of them) based on the requriement

Degree 2 consistency

18.9.1 Degree-Two Consistency

The purpose of **degree-two consistency** is to avoid cascading aborts without necessarily ensuring serializability. The locking protocol for degree-two consistency uses the same two lock modes that we used for the two-phase locking protocol: shared (S) and exclusive (X). A transaction must hold the appropriate lock mode when it accesses a data item, but two-phase behavior is not required.

In contrast to the situation in two-phase locking, S-locks may be released at any time, and locks may be acquired at any time. Exclusive locks, however, cannot be released until the transaction either commits or aborts. Serializability is not ensured by this protocol. Indeed, a transaction may read the same data item twice and obtain different results. In Figure 18.21, T_{32} reads the value of Q before that value is written by T_{33}, and again after it is written by T_{33}.

- The purpose of degree-two consistency is to avoid cascading aborts without necessarily ensuring seirializability.
- cascading abort → uncommitted dependency problem. (when source transactionf ailed, and the reader has to roll back)
- use same 2 lock mode (S lock, X lock)
    - 2PL is not require (S lock can be lock unlock anytime, but X- Lock has to be unlock at sync point(avoid cascading rollback)

Cursor stability

- A slightly different kind of level of consistency
- For reads, each tuple is locked, read, and lock is immediately released
- X-locks are held till end of transaction
- Special case of degree-2 consistency.

👉 Like as a user, you can choose isolation level

Read uncommitted → degree 1

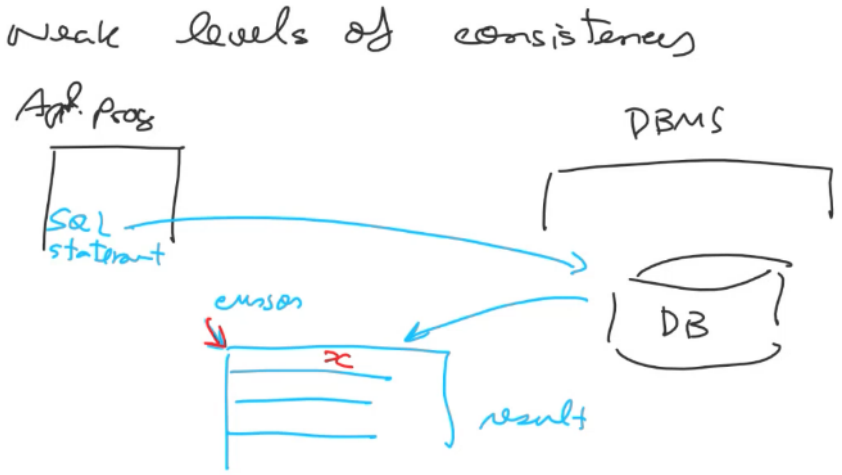
Read committed → degree 2 (snapshot isolation, cursor stability)

This is where degree 2 consistency come from.

> Cursor stability → popular algorithm for degree 2

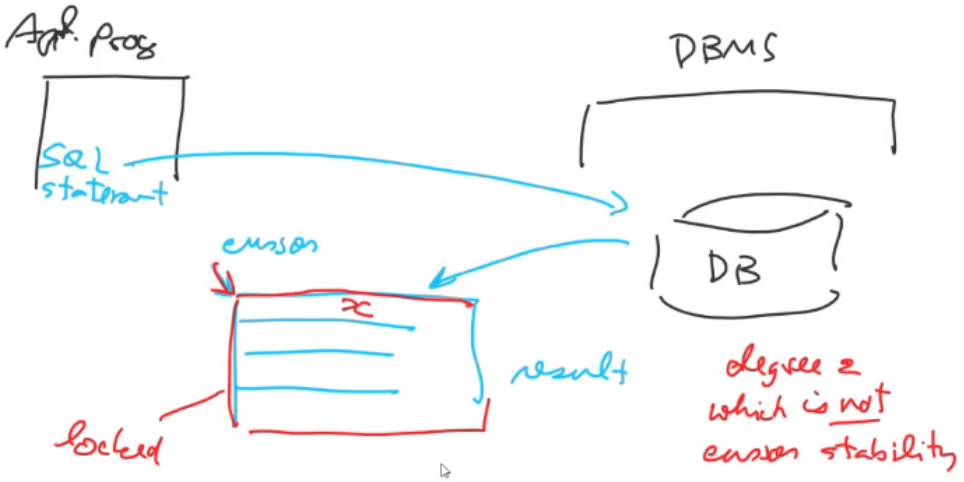
can you give an example of degree 2 consistency that is not cursor stability? snapshot isolation

- another is
  - (a way to implement degree 2 that is not cursor stability)



- you look at this result, you first see x, move cursor → will you always see this x, or is it possible that you see different x? → if repeatable read → always same value (old record, old value) (you may find new insert value)
- in the case → read committed (degree 2)
  - cursor stability → lock one row at a time, move forward can see value that younger write

Another one



- lock entire thing first
  - you move cursor, system unlock one by one.
- **what if we want to ensure serializability from this degree 2**
  - do not move the cursor backward. (only forward)
  - for cursor stability → you can move forward backward (you may want to see realtime change)